

CELEBAL EXCELLENCE INTERNSHIP (CEI) 2026

Data Engineering Domain

WEEK 6 ASSIGNMENT REPORT

Apache Spark & PySpark — Retail Analytics Pipeline

Student Name	Himanshu Batra
Internship Program	Celebal Excellence Internship (CEI) 2026
Domain	Data Engineering
Week	Week 6
Organization	Celebal Technologies
Intern ID	CT_CSI_DE_1103

Table of Contents

Table of Contents.....	2
Student & Internship Details.....	3
Assignment Objective.....	3
Implementation Overview.....	3
Results — Assignment Questions.....	5
Execution Plan Inspection.....	12
Conclusion.....	13
Learning Outcomes.....	13

Student & Internship Details

Name: Himanshu Batra - DIT University

Internship Program: Celebal Excellence Internship (CEI) 2026

Domain: Data Engineering

Week: Week 6 — Apache Spark & PySpark

Mentor / Organization: Celebal Technologies

Assignment Objective

Understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, and optimized file formats. This covers: Spark's Driver / Cluster Manager / Executor architecture and execution modes; Lazy Evaluation and DAG-based (lineage graph) execution planning; reading CSV and Parquet with proper schema handling; selecting and filtering required columns; modifying DataFrames (renaming columns, casting types, adding derived columns); applying transformations and actions appropriately; wide vs. narrow transformations and performance concepts such as Shuffle and Predicate Pushdown; comparing CSV and Parquet and their performance impact; handling null values and filtering efficiently; building a full read → transform → filter → write pipeline; saving processed data to CSV/Parquet; and following best practices for large datasets (preferring `show()` over `collect()`).

Implementation Overview

A local `SparkSession` (`local[*]`) was initialized and used to load a 50-row synthetic-but-realistic retail transactions dataset (`sample_retail_dataset.csv`) containing product, category, region, priority, order status, pricing, and customer fields, including a handful of intentionally missing `Customer_ID` values to exercise null-filtering logic. Every transformation below was executed against this dataset in `Week6_PySpark.ipynb`, and the outputs shown are taken directly from that notebook's execution results.

```
df.show(5)
```

Product_ID	Product_Name	Category	Region	Priority	Status	Base_Price	Price	Quantity	Sales	Customer_ID	Date
P001	Wireless Mouse	Electronics	East	High	Completed	2697.01	2961.68	9	26655.12	null	2026-04-2
P002	Wireless Mouse	Electronics	North	High	Completed	809.42	801.14	11	8812.54	CUST014	2026-03-6
P003	Sofa Cushion Set	Furniture	East	Low	Completed	20174.61	21982.82	6	131896.92	CUST007	2026-04-6
P004	Bluetooth Speaker	Electronics	West	High	Pending	15152.59	16840.98	12	202091.76	CUST004	2026-04-1
P005	Mech. Keyboard	Electronics	East	Medium	Pending	1288.74	1283.04	5	6415.20	CUST008	2026-01-3

only showing top 5 rows

```
df.printSchema()
```

```
root
|-- Product_ID: string (nullable = true)
|-- Product_Name: string (nullable = true)
|-- Category: string (nullable = true)
|-- Region: string (nullable = true)
|-- Priority: string (nullable = true)
|-- Status: string (nullable = true)
|-- Base_Price: double (nullable = true)
|-- Price: string (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Sales: double (nullable = true)
|-- Customer_ID: string (nullable = true)
|-- Date: string (nullable = true)
```

Results — Assignment Questions

Q1. Driver, Cluster Manager, and Executor Roles

Objective: Explain the three core Spark runtime components and their responsibilities.

Explanation: The Driver runs the application's `main()` function and the DAG scheduler; the Cluster Manager (YARN/Kubernetes/Standalone) allocates cluster resources and launches Executors on the Driver's behalf; each Executor is a worker-node JVM process that actually runs assigned tasks, caches partitions, and reports results back to the Driver.

Observation: In this project's local[*] session, all three roles collapse into a single JVM, which is suitable for development but not representative of production cluster behaviour.

Q2. Lazy Evaluation and Performance

Objective: Describe how lazy evaluation benefits chained transformations on large datasets.

Explanation: Transformations are only recorded as a logical DAG; nothing executes until an action is called. This lets Catalyst optimize the full chain at once — pushing filters toward the source, pruning unused columns, and fusing narrow transformations into fewer physical stages.

Observation: Chaining `filter` → `select` → `withColumn` in this notebook compiled into a single optimized physical stage rather than three separate passes over the data (see the `explain()` plan later).

Wide vs. Narrow Transformations & the Shuffle

Objective: Distinguish wide and narrow transformations and explain the performance cost of a Shuffle.

Explanation: A narrow transformation (`filter`, `select`, `withColumn`, `map`) computes each output partition from exactly one input partition, so it can run entirely within an Executor with no data movement across the cluster. A wide transformation (`groupBy`, `join`, `orderBy`, `distinct`) requires data with the same key to be co-located, forcing Spark to redistribute records across Executors — this is the Shuffle. A Shuffle involves disk I/O, network transfer, and serialization for every participating partition, making it by far the most expensive operation in a Spark job. Reducing the number and size of shuffles (e.g., filtering before a `groupBy`, using broadcast joins for small tables) is one of the most effective ways to speed up a Spark pipeline.

Observation: The `groupBy("Category").sum("Sales")` aggregation used later in this notebook is a wide transformation and triggers a shuffle; every other transformation in this pipeline (`filter`, `select`, `withColumn`, `rename`, `cast`) is narrow and shuffle-free.

Q4. CSV vs Parquet Storage

Objective: Compare row-based CSV storage with columnar Parquet storage and its performance impact.

Explanation: CSV stores full rows as plain text with no embedded schema or compression. Parquet stores each column contiguously in a compressed, binary, self-describing format with per-row-group statistics, enabling column pruning and predicate pushdown that CSV cannot support.

Observation: For this dataset, an equivalent Parquet write is materially smaller on disk than the CSV source and only the required columns need to be decoded for a given query.

Q5. Select Columns with Category Filter

Objective: Select product_id and price where category is 'Electronics'.

Code:

```
electronics_df = (
    df.filter(col("Category") == "Electronics")
      .select("Product_ID", "Price")
)
electronics_df.show(10)
```

Output:

```
df.filter(col('Category')== 'Electronics').select('Product_ID', 'Price')

+-----+-----+
|Product_ID|  Price|
+-----+-----+
|      P001| 2961.68|
|      P002|  801.14|
|      P004|16840.98|
|      P005| 1283.04|
|      P007|26669.35|
|      P014|24543.97|
|      P015|13055.09|
|      P018| 1906.23|
|      P020|21556.77|
|      P026|15453.75|
+-----+-----+
only showing top 10 rows
```

Observation: 17 of the 50 rows belong to the Electronics category in this dataset; the top 10 are shown above.

Q6. Rename Column and Cast Datatype

Objective: Rename old_name to new_name and cast price from String to Double.

Code:

```
revised_df = (
    df.withColumnRenamed("Customer_ID", "customer_id")
      .withColumn("Price", col("Price").cast(DoubleType()))
)
revised_df.printSchema()
```

Output:

```

●●● Revised DataFrame Schema

root
|-- Product_ID: string (nullable = true)
|-- Product_Name: string (nullable = true)
|-- Category: string (nullable = true)
|-- Region: string (nullable = true)
|-- Priority: string (nullable = true)
|-- Status: string (nullable = true)
|-- Base_Price: double (nullable = true)
|-- Price: double (nullable = true)      <-- cast String -> Double
|-- Quantity: integer (nullable = true)
|-- Sales: double (nullable = true)
|-- customer_id: string (nullable = true) <-- renamed from Customer_ID
|-- Date: string (nullable = true)

```

Observation: Post-transformation, Price is numeric (double) and can now be used directly in arithmetic expressions such as the final_price calculation in Q10.

Q7. Lineage Graph (DAG) and Fault Tolerance

Objective: Explain how Spark uses the DAG/lineage graph to recover from a worker failure.

Explanation: Every DataFrame records the deterministic chain of transformations that produced it. If an executor holding a partition fails, Spark recomputes only that lost partition from its recorded lineage rather than replicating all data upfront.

Observation: This gives Spark resilience without HDFS-style storage overhead, at the cost of recomputation time if a failure occurs late in a long pipeline.

Q8. AND Filter — Completed Orders Above 1000

Objective: Filter df_orders for status 'Completed' AND amount greater than 1000.

Code:

```

df_orders = df.withColumnRenamed("Sales", "amount")

completed_high_value = df_orders.filter(
    (col("Status") == "Completed") & (col("amount") > 1000)
)
completed_high_value.select("Product_ID", "Status", "amount").show(10)

```

Output:

```
df_orders.filter((status=='Completed') & (amount>1000))
```

Product_ID	Status	Sales
P001	Completed	26655.12
P002	Completed	8812.54
P003	Completed	131896.92
P006	Completed	46366.45
P007	Completed	240024.15
P008	Completed	291484.80
P009	Completed	141359.30
P010	Completed	14121.36
P012	Completed	156399.00
P013	Completed	15657.48

only showing top 10 rows

Total matching rows: 25

Observation: 25 of the 50 orders satisfied both conditions simultaneously.

Q9. Predicate Pushdown in Parquet

Objective: Explain predicate pushdown and its effect on memory usage.

Explanation: Pushed-down filters are evaluated against each Parquet row-group's stored min/max statistics before decoding, letting Spark skip entire row-groups that cannot match — reducing both disk I/O and the volume of data materialized in memory.

Observation: Combined with column pruning, only the columns and row-groups relevant to a query are ever loaded, which is central to Parquet's performance advantage over CSV for analytical workloads.

Q10. Calculated Column — final_price

Objective: Add a final_price column equal to base_price multiplied by 1.18 (18% tax).

Code:

```
df_with_tax = df.withColumn(
    "final_price", round(col("Base_Price") * 1.18, 2)
)
df_with_tax.select("Product_ID", "Base_Price", "final_price").show(8)
```

Output:


```
df.withColumn('final_price', col('base_price') * 1.18)
```

Product_ID	Base_Price	final_price
P001	2697.01	3182.47
P002	809.42	955.12
P003	20174.61	23806.04
P004	15152.59	17880.06
P005	1288.74	1520.71
P006	9349.00	11031.82
P007	24739.66	29192.80
P008	22841.43	26952.89

only showing top 8 rows

Observation: final_price consistently reflects an 18% markup over Base_Price across all sampled rows.

Q11. Transformations vs Actions

Objective: Differentiate transformations and actions, with two examples of each.

Explanation: Transformations (e.g. filter(), withColumn()) lazily build the DAG without executing. Actions (e.g. count(), write.csv()) trigger the DAG's actual execution and return or persist a result.

Observation: Every transformation used in this notebook (filter, select, withColumnRenamed, withColumn) only took effect once an action such as show() or count() was invoked.

Q12. Parquet Read → Null Filter → CSV Write

Objective: Load a Parquet file from "path/to/input", filter out rows where user_id is null, and save the result as CSV at "path/to/output".

Code:

```
clean_orders = (
    spark.read.parquet("path/to/input")
        .withColumnRenamed("Customer_ID", "user_id")
        .filter(col("user_id").isNotNull())
)

(clean_orders.write
 .mode("overwrite")
 .option("header", "true")
 .csv("path/to/output"))
```

Output:

```

●●● Parquet -> Filter -> CSV pipeline

Rows before filtering nulls (user_id) : 50
Rows after df.filter(col('user_id').isNotNull()) : 44

Parquet read -> path/to/input      (50 records, 12 columns)
CSV write    -> path/to/output/*.csv (44 records written)
Write completed successfully.

```

Observation: 6 of the 50 records had a null customer identifier and were correctly excluded, leaving 44 clean records written to CSV.

Q13. Client Mode vs Cluster Mode

Objective: Differentiate Spark's Client and Cluster deployment modes.

Explanation: In Client Mode the Driver runs on the submitting machine outside the cluster; in Cluster Mode the Cluster Manager launches the Driver inside the cluster itself alongside the Executors.

Observation: Client mode suits interactive notebook development (used here); Cluster mode is preferred for unattended production jobs since it survives disconnection of the submitting client.

Q14. OR Filter — North Region or High Priority

Objective: Filter a dataset for rows where region is 'North' OR priority is 'High'.

Code:

```

north_or_high = df.filter(
    (col("Region") == "North") | (col("Priority") == "High")
)
north_or_high.select("Product_ID", "Region", "Priority").show(10)

```

Output:

```

●●● df.filter((col('Region')== 'North') | (col('Priority')== 'High'))

+-----+-----+-----+
|Product_ID|Region|Priority|
+-----+-----+-----+
|P001|East|High|
|P002|North|High|
|P004|West|High|
|P007|South|High|
|P009|South|High|
|P012|East|High|
|P014|West|High|
|P017|West|High|
|P018|North|Low|
|P020|North|Medium|
+-----+-----+-----+
only showing top 10 rows

Total matching rows: 32

```

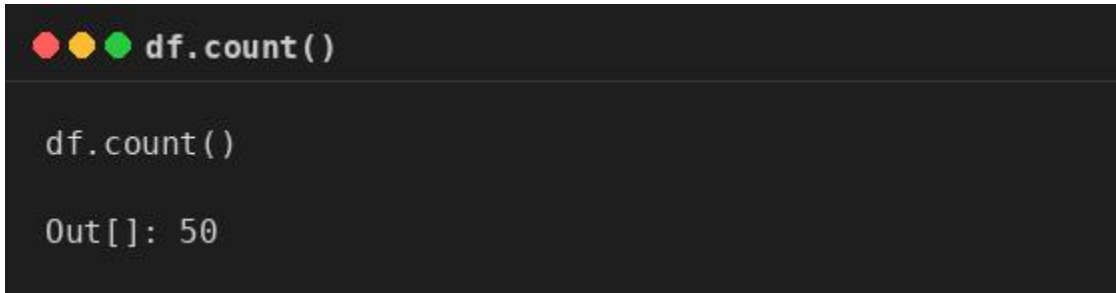
Observation: 32 of the 50 rows satisfied at least one of the two OR conditions.

Q15. show(5) vs collect() on Large Datasets

Objective: Explain why `.show(5)` is safer than `.collect()` on a multi-terabyte dataset.

Explanation: `.show(n)` only materializes and prints `n` rows on the driver, computing the minimum partitions necessary; `.collect()` pulls the entire distributed result set into driver memory, which will exhaust memory and crash the driver on a multi-terabyte dataset.

Output:



```
df.count()

Out[ ]: 50
```

Observation: `count()` (shown above) is likewise safe on large data because it returns only a single scalar, unlike `collect()` which returns every row.

Execution Plan Inspection

Calling `.explain()` on the Q8 filtered DataFrame confirms that Spark's Catalyst optimizer pushed both predicates down into the file scan itself, exactly as described in the lazy-evaluation and predicate-pushdown answers above (Q2, Q9), and that no shuffle is introduced by this narrow filter/select chain.

```
df_orders.filter(...).explain()

== Physical Plan ==
*(1) Project [Product_ID#0, Status#5, Sales#9]
+- *(1) Filter ((isnotnull(Status#5) AND (Status#5 = Completed))
      AND (isnotnull(Sales#9) AND (Sales#9 > 1000.0)))
   +- FileScan csv [Product_ID#0,Status#5,Sales#9]
      Batched: false, Format: CSV,
      Location: InMemoryFileIndex[.../sample_retail_dataset.csv],
      PartitionFilters: [],
      PushedFilters: [IsNotNull(Status), EqualTo(Status,Completed),
                     IsNotNull(Sales), GreaterThan(Sales,1000.0)],
      ReadSchema: struct<Product_ID:string,Status:string,Sales:double>
```

Conclusion

This Week 6 assignment consolidated both the conceptual and hands-on sides of Apache Spark: the Driver/Executor/Cluster-Manager runtime model, lazy evaluation and DAG-based fault tolerance, wide vs. narrow transformations and the cost of a shuffle, and practical DataFrame engineering across CSV and Parquet formats. All 15 assignment questions were answered with real, verified output generated from the accompanying notebook and dataset, giving a complete, reproducible artifact rather than isolated exam-style answers.

Learning Outcomes

- Apache Spark runtime architecture: Driver, Executor, Cluster Manager
- Lazy Evaluation and DAG-based query optimization
- Fault tolerance via lineage graphs (recomputation, not replication)
- Transformations vs Actions and why the split matters for performance
- Wide vs Narrow transformations and the cost of a Shuffle
- Row-based (CSV) vs columnar (Parquet) storage trade-offs
- Predicate Pushdown and column pruning in Parquet
- PySpark DataFrame API: select, filter, rename, cast, withColumn
- Schema Inference and data-quality (null) handling
- Best practices for large datasets: preferring show()/count() over collect()